Chapter 2 · Relational algebra and tuple calculus

Suppose a college has Student(StudentID, Name), Enrolment(StudentID, CourseID) and Required(CourseID). We want the students enrolled in every required course. First solve it by checking each student's checklist. The symbols should describe that same check.

Vocabulary. Selection keeps rows satisfying a predicate. Projection keeps specified columns and removes duplicate tuples in classical algebra. Product forms all pairs. A theta join keeps pairs satisfying its condition. A natural join matches on every shared attribute name. Rename makes roles and attribute names explicit. Union, intersection and difference require compatible schemas. Division expresses an every-member requirement.

R04 Evaluate an algebra expression

1. Parse the parentheses and draw the expression tree.
2. Write the output attributes at every node.
3. Evaluate the innermost operations and show intermediate rows.
4. Remove duplicates after projection and other set operations.
5. Apply the final projection and count the resulting tuples.

For a natural join on (P,Q), matching P alone is insufficient. Same-named columns can make a join more restrictive than the story suggests.

Worked example W04. Take R(Student,Course)={(Asha,Math),(Asha,Physics),(Ravi,Math)} and S(Course,Room)={(Math,10),(Physics,20)}. R natural-join S matches Course. Its rows are (Asha,Math,10), (Asha,Physics,20), (Ravi,Math,10). Selection Room=10 leaves the first and third rows. Projection Student returns {Asha,Ravi}. Projection Course instead returns just {Math}: the two identical projected values become one tuple. To see the other operators, rename two copies of Student as S1 and S2 before taking their product; a two-row relation produces four ordered pairs. Union combines compatible tuple sets, intersection keeps common tuples, and difference keeps tuples in the first set but not the second.

Check before moving on. Write the attributes at every expression-tree node. A natural join uses every shared attribute name, and projection uses sets.

R05 Join cardinality and bounds

For an equality join on J, count rows per join value: the result size is the sum of nR(j) times nS(j). If J is a key on one side, each row on the other side has at most one match. A valid non-null foreign key to that key guarantees exactly one match per child. A nullable reference does not. For products, multiply input sizes. For set projections, count distinct projected tuples.

Worked example W05. R has join-value counts a:2 and b:1. S has counts a:3, b:2 and c:1. There are 2×3+1×2=8 matching pairs. The unmatched c contributes zero. If S's join column becomes unique, each R row has at most one match, so the join has at most 3 rows. A non-null foreign key from R to S makes that upper bound exact. The frequency calculation and the key-based bound answer different versions of the same question.

Check before moving on. Do duplicates describe distinct tuples with the same join value? Keep every matching pair before the final set projection.

R06 Translate some none and every

- Some: at least one matching row satisfies the condition.
- None: no matching row satisfies the condition.
- Every: no relevant row fails the condition.

For R(StudentID,CourseID) divided by Required(CourseID):

1. Take candidate students from the projection of R.
2. For each candidate, form the set of courses associated with it.
3. Keep it if Required is a subset of that set.

Empty Required makes the condition true for every candidate from the specified candidate domain. It does not invent students missing from that domain. SQL implementations using NOT EXISTS need explicit NULL reasoning.

Worked example W06. Enrolment={(1,M),(1,P),(2,M)} and Required={M,P}. Student 1 covers both required values; student 2 misses P. Division returns {1}. Using candidate set C={1,2}, first form C×Required: (1,M),(1,P),(2,M),(2,P). Subtract Enrolment to get the missing pair (2,P). Project its student to obtain {2}; C minus {2} is {1}. If Required is empty, there are no missing pairs and division returns {1,2}. If candidates instead come from Student={1,2,3}, a NOT EXISTS query returns all three. The candidate domain must be stated.

Check before moving on. Rewrite every as no required value is missing. Test an empty requirement and an absent candidate.

R07 Query equivalence and counterexamples

Use when: two expressions are claimed to return the same result.

Track schemas, matching columns and quantifiers. Try a tiny input containing duplicates, an unmatched row, two matches or an empty relation. One differing output disproves equivalence. Matching on a few examples does not prove equivalence. To justify a valid rewrite, use an operator law with its conditions; for example, pushing a selection into an input requires that input to contain all referenced attributes.

Worked example W07. Consider the claim that Team is contained in Defender intersection Forward whenever it is contained in Defender union Forward. Let Team={Asha}, Defender={Asha}, Forward be empty. Asha belongs to the union but not the intersection. One row disproves the claim. A valid pushdown example is selection Room=10 on R(Student,Course) joined with S(Course,Room): the selection can move into S because S supplies Room. Moving it into R is not well-formed because R has no Room attribute. A counterexample disproves a claimed law; several matching examples do not prove it.

Check before moving on. Try one missing match or one extra tuple. For a claimed valid rewrite, state the operator law and its conditions.

R08 Evaluate tuple relational calculus

A predicate is a condition that can be true or false. A tuple variable stands for a row. An existential quantifier means at least one witness; a universal quantifier means every row in its stated domain.

1. Identify the output tuple and its domain.
2. Give each quantified variable a separate role.
3. For each candidate output tuple, test the conditions and search for witnesses.
4. Count qualifying output tuples, not witness combinations.

Translate a universal condition using “no counterexample exists.” Do not assume two variables are distinct unless required. An existential condition may be redundant because the candidate tuple itself supplies a witness. Safe calculus queries depend on the database's active domain and fixed query constants; finite output alone is not an adequate general definition of safety.

Worked example W08. Let X(P,Q)={(1,a),(2,b),(3,c)} and Y(P,S)={(1,10),(1,20),(2,30),(3,1)}. The predicate for an output row x is: x belongs to X and some y in Y has y.P=x.P and y.S>1. x=(1,a) has two witnesses, x=(2,b) has one, and x=(3,c) has none. There are 2 output tuples, not 3 witnesses. Adding a condition that some z in X has z.P=x.P changes nothing: z=x already works. In notation, $\{x \mid x \in X \wedge \exists y \in Y\ (y.P = x.P \wedge y.S > 1)\}$ states precisely that row test. Unrestricted $\{x \mid x \notin X\}$ is not safe over an unbounded tuple universe.

Check before moving on. Separate output tuples from witnesses. Do not assume two variable names force two different rows.

Previous-year questions, worked

Read the original question and options in the linked official paper. These are original explanations, not replacement question statements. PDF page numbers count from the first page of the downloaded file.

DA-2024-Q26 · Containment and union

Official question, PDF page 16. Published key: **C**.

Let T,D,F be the projected name sets. Every Team name must appear in at least one of D,F, so T must be a subset of D union F. Test this by checking whether T minus (D union F) is empty. That is C. A demands both roles, which is too strong. B and D reverse the inclusion: they can reject an extra player even when all Team names are covered.

DA-2025-Q17 · Read joins as a sentence

Official question, PDF page 12. Published key: **C**.

Filter makers to ABC, join cars on model, and filter those cars to red. Join ownership on serial. The final projection contains owners of red ABC cars: C. There is no count or requirement that an owner have several cars. Both filters are required, not an OR. The output is owner identifiers, not car rows.

DA-2025-Q62 · Division

Official question, PDF page 32. Published key: **1 to 1**.

List the four required branches, then make one branch set for each account holder from the displayed ownership tuples. Only Karteek covers all four. Extra accounts at a covered branch do not replace a missing branch. The divided relation therefore contains one owner. Use the original displayed tables to fill the four-column checklist before counting.

DA-2026-Q42 · Nested natural joins

Official question, PDF page 27. Published key: **D**.

X natural-join Y matches both P and Q. After the left selection, the surviving tuple is (P3,Q3,R2,S1). On the right, Y natural-join Z followed by S>1 leaves tuples with P1 and S2 or S5. The outer natural join shares P,Q,S. The left P3 tuple has no right partner. Its result is empty, so projection also has zero rows: D. Matching only P in the first join would already be wrong.

DA-2026-Q59 · Witnesses and an official range

Official question, PDF page 38. Published key: **2 to 3**.

For the printed condition S>1, the X rows with P1 and P2 qualify. P1 has two Y witnesses but still contributes one output tuple. P3's Y value is 1, so it fails the strict inequality. The additional X witness can be the output tuple itself. This derives 2. The published accepted range is 2 to 3; retain that range when grading the historical exam. This guide does not change >1 into ≥1 to manufacture the other endpoint.

CS-2024-set1-Q35 · Join then filter

Official question, PDF page 21. Published key: **2 to 2**.

Join on A. The matched triples are (10,20,90), (30,40,45), and (30,50,45). Apply B<C: the first two pass and the third fails. A=20 and A=50 have no S partner; A=40 has no R partner. The final count is 2.

CS-2024-set2-Q45 · Three distinct witnesses

Official question, PDF page 31. Published key: **B**.

Use three renamed copies of the person relation. Three copies require two binary Cartesian-product operations. Require all three pairwise identifier inequalities as well as the shared target condition. Without inequalities, one person could occupy all three roles. The option with two cross products is B.

CS-2025-set1-Q39 · Bind calculus variables

Official question, PDF page 39. Published key: **A**.

The output pname must come from a player p. A team t must exist with p.tid=t.tid and t.tname='MI'. A binds both variables to the right relations and includes the link. B and D give variables attributes their relations do not have. C omits the identifier match and would admit every player whenever any MI team exists.

CS-2022-Q25 · Division and missing pairs

Official question, PDF page 20. Published key: **A; B**.

A divides the ownership pairs by all Brand identifiers. B removes from the ownership candidate set anyone with a missing candidate×Brand pair. Both express the same requirement. C only asks for brands already appearing in Own and can omit an unowned Brand. D builds a product that ignores each employee's actual ownership. The candidate domain is employees occurring in Own. To include employees with no ownership when Brand is empty, a different expression must explicitly use Employee as its candidate domain.

Chapter practice

Original questions. Write the answer and the intermediate reasoning. Each lettered part belongs to the same problem. The separate solutions guide contains hints, steps and a final check.

P04 · NAT plus variation

X(P,Q)={(1,a),(2,a),(3,b)}, Y(Q,R)={(a,9),(a,10),(b,10)}. (a) List X natural-join Y, then select R=10 and project Q. How many tuples remain? (b) Remove (b,10) from Y and repeat. What is the product size |X×Y| in each case?

P05 · NAT plus variation

Distinct R tuples have join values {1,1,2,3}. Distinct S tuples have join values {1,1,1,2}. (a) Count equality-join pairs. (b) Keep just one S tuple per join value. Recount. Explain why an upper bound of |R| failed in (a).

P06 · Short answer

Enrolment={(1,M),(1,P),(2,M)}, Student={1,2,3}. (a) Divide Enrolment by Required={M,P}. (b) Make Required empty. Compare classical division, whose candidates come from Enrolment, with a no-missing-course query whose candidates come from Student.

P07 · Counterexample

(a) Give a one-name instance where Team minus (Defender union Forward) is empty but Team minus (Defender intersection Forward) is nonempty. (b) Give an instance where (Defender union Forward) minus Team is empty but the required Team coverage fails. Explain what each reversed containment tests.

P08 · NAT plus variation

X={(1,a),(2,b),(3,c)} with attributes (P,Q); Y={(1,10),(1,20),(2,30),(3,1)} with attributes (P,S). Output each x in X for which some y in Y has the same P and S>1. (a) Count output tuples and successful (x,y) witnesses. (b) Change >1 to ≥1. Recompute. Does an additional $\exists z \in X$ with z.P=x.P change the output?

Before the exam

Translate the requested output into rows, sets or witnesses before counting. Keep intermediate attributes visible. Test every/none expressions on an empty requirement. For equivalence claims, a small counterexample is often faster than a large calculation.